doc: prd status: approved

AltPass — Product Requirements

AltPass turns an inaccessible PDF into an accessible HTML document and shows the proof; it is for the people who publish documents under accessibility regulation (public-sector comms teams, universities, EU online businesses). Source: `scope.md > The Unique Kernel`, `scope.md > Who It's For`.

The Core Journey

1. Amara opens `http://localhost:3000`. The hero shows a before/after teaser and a one-line promise: "Make any PDF pass."
2. She drops a PDF onto the upload zone — or clicks one of three sample PDFs. (`scope.md > The Core Loop`)
3. A progress stepper shows what's happening: *Reading the PDF → Rebuilding structure → Writing alt text → Checking with axe-core*.
4. The result view appears: a verdict banner (score before → after, axe pass rate before → after, number of issues fixed, number of items to review).
5. She explores the **before/after** side by side: left = the original PDF pages as sighted users see them and a "what a screen reader announces" transcript; right = the accessible HTML (live, in a sandboxed frame) with a "document outline" view.
6. She reads the **checklist of issues fixed** (each with WCAG criterion and what was done) and the **needs review** list (draft alt text, flagged items).
7. She edits an alt text; the preview updates.
8. She downloads the accessible HTML (standalone file) — and the audit as JSON.
9. She uploads the next PDF.

Screens and Layout

Single page, three stacked zones that reveal as she goes:

- **Hero + upload** — headline, before/after teaser, drop zone, sample chips, "why now" legal strip (EU Accessibility Act, ADA Title II dates).
- **Result** — verdict banner → before/after compare → fixes checklist → alt-text review table → download bar.
- **Footer** — how it works, limits and honesty note (what's heuristic), license.

Look and Feel

Warm paper (#FAF7F2) and ink (#15171C); Fraunces for display, Inter for UI; generous spacing, rounded 16px cards, soft shadow; "pass" green (#0F9D6B) and "issue" vermilion (#D9482B); score rings drawn in SVG; subtle stamp motif for the verdict ("PASS" / "NEEDS REVIEW"). The app's own UI must itself be accessible (axe clean, keyboard operable, visible focus, `prefers-reduced-motion`, dark mode). Source: `scope.md > Inspiration & Identity`.

Features and Behavior

Upload and samples

- Drag-and-drop or file picker; PDF only, ≤ 15 MB, ≤ 30 pages (later pages are skipped with a notice).
- Three sample PDFs available as one-click chips: *Council meeting agenda*, *Product launch slide deck*, *Spring price list*.
- Errors are explicit: not a PDF, encrypted, no extractable text (scanned) → a clear message with next steps.

Structure rebuilding

- Detect title, headings (levels by relative font size/weight), paragraphs, bullet/numbered lists, simple tables (aligned columns), links (annotations).
- Fix reading order for two-column pages; drop running headers/footers and page numbers; keep page-break markers.
- Detect document language from text.

Alt text

- Extract embedded images; draft alt text with the configured adapter; mark each as `alt: draft` and list it for review; allow marking an image decorative (`alt=""`).
- The UI states which adapter produced the text ("Rule-based fallback" vs "Vision model: ").

Audit, score and proof

- Before: PDF-level checks (tagged, title, language, text extractable, headings exposed, reading order risk, images with alt, tables with headers, link text, bookmarks). Weighted score 0–100.
- After: the same checks against the output, plus axe-core results on the output; pass rate = passes / (passes + violations) for rules that applied.
- axe-core is also run on a naive PDF→HTML conversion (no structure) as the "before" axe number.

Output

- Standalone accessible HTML: `lang`, `<title>`, skip link, landmarks, heading outline + table of contents, figures with alt, tables with `<th scope>`, readable CSS (min 4.5:1 contrast, reflow at 320 px, dark mode, reduced motion).

- As Amara, I want a single score and a list of what was fixed so that I can put evidence in my compliance file.
    - [ ] Result view shows score before and after, and axe pass rate before and after.
    - [ ] Each fixed issue names its WCAG success criterion.
- As Amara, I want to correct AI-drafted alt text so that I stay responsible for what is published.
    - [ ] Editing an alt text updates the preview and the download.

States and Boundaries

- **First use** — hero + upload + samples; no results.
- **Processing** — stepper; upload disabled.
- **Success** — result view.
- **No LLM key** — rule-based alt text; banner says alt text is drafted from context and must be reviewed.
- **Scanned PDF / encrypted / too big** — specific error message.
- **Persistence** — results held in server memory for 1 hour; nothing stored on disk after the process ends.

Product Decisions

- HTML output, not remediated PDF — HTML reflows, works with every screen reader and is verifiable with axe-core.
- Pluggable LLM adapter with an honest rule-based default — the app must work with zero keys and zero network.
- Local-first: documents never leave the machine unless an LLM key is configured.
- Pre-generated sample PDFs so the demo is reproducible.

What We're Building

Everything under "Features and Behavior", the three samples, a CLI (`npm run audit -- file.pdf`), tests, and a README.

Deferred From the POC

OCR, PDF/UA output, batch upload, accounts/history, deployment — see `scope.md > Later`.

Possible Later Enhancements

Source-PDF colour contrast checks; form-field labelling; per-language alt text; a CI action that fails a build on inaccessible PDFs.

Non-Goals

- Not a legal certification.
- Not editing the original PDF.
- Not storing documents.

Open Questions

- Which vision model to default to once a key is available (can wait; adapter is configurable).